

DM : Calcul des séquents propositionnel

Le but de ce TP est d'étudier l'autre système déductif dû à Gentzen (mentionné en cours) : le calcul des séquents, et d'implémenter la recherche de preuve dans ce système, en langage **OCaml**. On se limite ici à sa version propositionnelle, c'est-à-dire sans quantificateurs.

A Système déductif du calcul des séquents

A.1 Introduction

On étend la notion de séquents pour permettre à la fois un ensemble de formules en contexte (à gauche du $\vdash$) et un ensemble de formules dans la conclusion (à droite du $\vdash$).

Intuitivement, la partie gauche du séquent est interprétée comme une conjonction (on a *toutes* ces hypothèses) et la partie droite comme une disjonction (on a *au moins une* de ces conclusions). Mathématiquement :

Définition 1

Validité d'un séquent (dans le calcul des séquents) :

Le séquent $\Gamma \vdash \Delta$ est **valide**, noté $\Gamma \models \Delta$, si toute valuation v satisfaisant *toutes* les formules de Γ satisfait également *au moins une* formule de Δ .

Le calcul des séquents vise à corriger les défauts suivants de la déduction naturelle :

- un manque de symétrie, qui donne des règles peu naturelles – par exemple pour l'élimination du $\vee$.
- la possibilité d'avoir des formules arbitrairement compliquées en étape intermédiaire dans la preuve d'un séquent « simple », ce qui rend difficile la recherche de preuves et le raisonnement sur ces preuves.
- le fait que certaines règles ne sont pas inversibles – leur conclusion peut être prouvable sans que leurs prémisses ne le soient – la règle d'introduction du $\vee$, par exemple.

A.2 Règles du système déductif

$$\begin{array}{c}
 \frac{}{\Gamma, \phi \vdash \phi, \Delta} \text{ (Ax)} \\
 \\
 \frac{\Gamma \vdash \Delta, \phi \quad \Gamma', \phi \vdash \Delta'}{\Gamma, \Gamma' \vdash \Delta, \Delta'} \text{ (Cut)} \\
 \\
 \frac{}{\Gamma, \perp \vdash \Delta} \perp \\
 \\
 \frac{\Gamma, \phi, \psi \vdash \Delta}{\Gamma, \phi \wedge \psi \vdash \Delta} (\wedge \vdash) \\
 \\
 \frac{\Gamma, \phi \vdash \Delta \quad \Gamma, \psi \vdash \Delta}{\Gamma, \phi \vee \psi \vdash \Delta} (\vee \vdash) \\
 \\
 \frac{\Gamma \vdash \phi, \Delta \quad \Gamma, \psi \vdash \Delta}{\Gamma, \phi \rightarrow \psi \vdash \Delta} (\rightarrow \vdash) \\
 \\
 \frac{\Gamma \vdash \phi, \Delta}{\Gamma, \neg \phi \vdash \Delta} (\neg \vdash) \\
 \\
 \frac{}{\Gamma \vdash \top, \Delta} \top \\
 \\
 \frac{\Gamma \vdash \phi, \Delta \quad \Gamma \vdash \psi, \Delta}{\Gamma \vdash \phi \wedge \psi, \Delta} (\vdash \wedge) \\
 \\
 \frac{\Gamma \vdash \phi, \psi, \Delta}{\Gamma \vdash \phi \vee \psi, \Delta} (\vdash \vee) \\
 \\
 \frac{\Gamma, \phi \vdash \psi, \Delta}{\Gamma \vdash \phi \rightarrow \psi, \Delta} (\vdash \rightarrow) \\
 \\
 \frac{\Gamma, \phi \vdash \Delta}{\Gamma \vdash \neg \phi, \Delta} (\vdash \neg)
 \end{array}$$

FIGURE III.1 – Règles du calcul des séquents propositionnel

Seule la règle de coupure présente les mêmes problèmes que la déduction naturelle. Elle ne possède pas de règle d'élimination. Toutes les autres règles sont inversibles.

La force du système vient du fait qu'on peut prouver un théorème d'**élimination des coupures** : si un séquent est dérivable en utilisant la règle de coupure, il l'est également sans utiliser cette règle. Ce point est le cœur de la démonstration de Gentzen, qui est hautement non triviale dans le cas général. Dans le cas propositionnel, en revanche, nous allons voir qu'il est possible de prouver directement la complétude du système sans coupure.

Commençons par deux petits exemples pour manipuler un peu ce système :

1. Prouver $\vdash p \vee \neg p$ avec le calcul des séquents.
2. Prouver $\vdash ((p \rightarrow q) \rightarrow p) \rightarrow p$ avec le calcul des séquents.

On admet pour l'instant que le calcul des séquents est un système déductif :

- **correct** : si $\Gamma \vdash \Delta$ est prouvable, alors $\Gamma \models \Delta$.
- **complet** : si $\Gamma \models \Delta$, alors $\Gamma \vdash \Delta$ est prouvable (et même sans coupure)

Ces résultats seront prouvés en dernière partie de TP.

B Recherche de preuve

Le but de cette section est d'implémenter une stratégie de *recherche de preuve* en OCaml. L'idée est de chercher une preuve (sans coupure) en se laissant guider par la syntaxe :

- si l'on trouve une preuve, alors le séquent est valide (**correction**).
- si l'on n'en trouve pas, alors le séquent n'est pas valide (**complétude**). Pour cela, il nous faudra aussi prouver la « complétude » de notre stratégie de recherche (si l'on ne trouve pas de preuve sans coupure en la suivant, c'est qu'il n'existe pas de preuve sans coupure du séquent).

B.1 Représentation OCaml

On se donne le type `'a prop` suivant pour représenter les formules propositionnelles :

```
OCaml
1 type 'a prop =
2   | Top
3   | Bot
4   | V of 'a (* variable propositionnelle *)
5   | Not of 'a prop
6   | And of 'a prop * 'a prop
7   | Or of 'a prop * 'a prop
8   | Impl of 'a prop * 'a prop
```

Dans la suite, il sera plus pratique de séparer le contexte Γ (resp. la conclusion Δ) en deux parties :

- une partie `gamma_var` ne contenant que des formules réduites à une variable, à `Bot` ou à `Top` ;
- une partie `gamma` pouvant contenir des formules arbitraires (y compris réduites à une variable ou une constante).

On se donne donc le type `'a sequent` suivant :

```
OCaml
1 type 'a sequent = {
2   gamma : 'a prop list;
3   gamma_var : 'a prop list;
4   delta : 'a prop list;
5   delta_var : 'a prop list;
6 }
```

B.2 Implémentation des règles de base

3. Écrire une fonction `create_sequent : 'a prop list -> 'a prop list -> 'a sequent` telle que `create_sequent l_gamma l_delta` renvoie un `'a sequent` dont les champs `gamma_var` et `delta_var` sont des listes vides, et les champs `gamma` et `delta` contiennent les listes passées en arguments.

4. Écrire une fonction `member : 'a -> 'a list -> bool` qui teste si un élément est dans une liste.
On interdit bien entendu l'utilisation de la fonction OCaml `List.mem` pour cette question.
5. Écrire une fonction `bot : 'a sequent -> bool` qui teste si la règle ($\perp$) peut être appliquée au séquent pris en argument.
Attention : il faut chercher dans `gamma` et dans `gamma_var`.
6. Écrire une fonction `top : 'a sequent -> bool` qui teste si la règle ($\top$) peut être appliquée au séquent pris en argument.
Attention : il faut chercher dans `delta` et dans `delta_var`.
7. Écrire un fonction `axiom : 'a sequent -> bool` qui teste si la règle (Ax) est applicable au séquent pris en argument.
Attention : la formule à trouver peut être dans `gamma` ou `gamma_var`, et dans `delta` ou `delta_var`.
Indication : On ne cherchera pas à faire un programme efficace pour cette fonction. Ne cherchez pas à "trier" les ensembles, par exemple.

B.3 Implémentation de la recherche de preuve

La stratégie de preuve proposée pour la suite est très simple. Si on peut appliquer (Ax) ou ($\perp$) ou ($\top$), on l'applique et on a prouvé le séquent. Sinon :

- Si `gamma` n'est pas vide, on regarde la première formule de la liste :
 - si c'est une variable ou une constante $\perp$ ou $\top$, on l'enlève de `gamma` et on le rajoute dans `gamma_var` ;
 - sinon, c'est une formule ayant un connecteur logique : on applique la règle (de gauche) correspondante, et on continue la recherche de preuve sur la ou les prémisses ;
- Sinon (`gamma` est vide), on procède de manière similaire avec `delta` (avec la règle de droite correspondante).
- Si `gamma` et `delta` sont vides, et que les règles (Ax), ($\perp$) et ($\top$) ne s'appliquent pas, alors aucune règle ne s'applique, et le séquent n'est pas valide.

B.3.i : Règles d'inférence : une étape

Commençons par implémenter chacune des règles par une fonction OCaml. Pour chaque règle, on prend en argument le séquent à démontrer et on renvoie le ou les séquents prémisses à démontrer pour en déduire une dérivation du séquent grâce à la règle.

Si la première formule de `gamma` (resp. `delta`) n'est pas celle sur laquelle on peut appliquer la règle considérée, on lèvera l'exception suivante :

OCaml

```
1 exception Wrong_rule of string
```

8. Écrire une fonction `and_gamma : 'a sequent -> 'a sequent` qui renvoie la prémisses de la règle ($\wedge \vdash$) appliquée à la première formule du champ `gamma` du séquent pris en argument. Autrement dit, si la première formule de `gamma` est de la forme $\phi \wedge \psi$, le séquent renvoyé par la fonction `and_gamma` devra contenir ϕ, ψ dans son contexte.
On lèvera l'exception `Wrong_rule "And Gamma"` si cette formule n'est pas une conjonction.
9. Écrire une fonction `or_gamma : 'a sequent -> 'a sequent * 'a sequent` qui renvoie les prémisses de la règle ($\vee \vdash$) appliquée à la première formule du champ `gamma` du séquent pris en argument (on renvoie ici deux séquents à prouver pour déduire le séquent initial).
On lèvera l'exception `Wrong_rule "Or Gamma"` si cette formule n'est pas une disjonction.
10. Écrire une fonction `impl_gamma : 'a sequent -> 'a sequent * 'a sequent` qui renvoie les prémisses de la règle ($\rightarrow \vdash$) appliquée à la première formule du champ `gamma` du séquent pris en argument.
On lèvera l'exception `Wrong_rule "Impl Gamma"` si cette formule n'est pas une implication.
11. Écrire une fonction `not_gamma : 'a sequent -> 'a sequent` qui renvoie la prémisses de la règle ($\neg \vdash$) appliquée à la première formule du champ `gamma` du séquent pris en argument.
On lèvera l'exception `Wrong_rule "Not Gamma"` si cette formule n'est pas une négation.
12. Écrire une fonction `and_delta : 'a sequent -> 'a sequent * 'a sequent` qui renvoie les prémisses de la règle ($\vdash \wedge$) appliquée à la première formule du champ `delta` du séquent pris en argument.
On lèvera l'exception `Wrong_rule "And Delta"` si cette formule n'est pas une conjonction.

13. Écrire une fonction `or_delta : 'a sequent -> 'a sequent` qui renvoie la prémisse de la règle $(\vdash \vee)$ appliquée à la première formule du champ delta du séquent pris en argument.
On lèvera l'exception `Wrong_rule "Or Delta"` si cette formule n'est pas une disjonction.
14. Écrire une fonction `impl_delta : 'a sequent -> 'a sequent` qui renvoie la prémisse de la règle $(\vdash \rightarrow)$ appliquée à la première formule du champ delta du séquent pris en argument.
On lèvera l'exception `Wrong_rule "Impl Delta"` si cette formule n'est pas une implication.
15. Écrire une fonction `not_delta : 'a sequent -> 'a sequent` qui renvoie la prémisse de la règle $(\vdash \neg)$ appliquée à la première formule du champ delta du séquent pris en argument.
On lèvera l'exception `Wrong_rule "Not Delta"` si cette formule n'est pas une négation.

B.3.ii : Recherche automatique de preuve

16. Écrire une fonction `proof_search : 'a sequent -> bool` qui renvoie `true` si le séquent pris en argument est valide, et `false` sinon, en implémentant la stratégie présentée précédemment.
17. Tester votre fonction sur des exemples : ceux du sujet sont fournis dans le fichier squelette, mais vous pouvez aussi tester d'autres séquents vus en cours.

C Propriétés théoriques du calcul des séquents

C.1 Quelques exemples supplémentaires

18. Prouver $\neg(p \vee q) \vdash \neg p \wedge \neg q$ avec le calcul des séquents.
19. Prouver $\neg p \wedge \neg q \vdash \neg(p \vee q)$ avec le calcul des séquents.

C.2 Correction

Le but de cette section est de montrer que le calcul des séquents est un système de preuve **correct** :

Théorème 1

Correction du calcul des séquents pour le calcul propositionnel :

Si $\Gamma \vdash \Delta$ est prouvable, alors $\Gamma \models \Delta$.

Une règle est dite **correcte** lorsque : si toutes les prémisses de la règle sont des séquents valides, alors le séquent conclusion de la règle est valide.

20. Pour chaque règle du calcul des séquents, montrer qu'elle est correcte. *On pourra se contenter de le prouver pour les règles $(\perp)$, $(\rightarrow \vdash)$ et $(\vdash \rightarrow)$ et admettre qu'il en va de même pour les autres.*
21. En déduire que le calcul des séquents est un système de preuve correct.

C.3 Complétude

Le but de cette section est de montrer que le calcul des séquents, **sans la règle (Cut)**, est un système de preuve **complet** :

Théorème 2

Si $\Gamma \models \Delta$, alors $\Gamma \vdash \Delta$ est prouvable sans coupure.

Dans cette section, et sauf mention explicite du contraire, si l'on parle des « règles du calcul des séquents », on exclura toujours la règle de coupure.

Pour cela, on va montrer que toutes les (autres) règles du calcul des séquents ont une propriété remarquable (que n'ont pas toutes les règles de la déduction naturelle).

Définition 2

Règle inversible :

Une règle d'inférence est dite **inversible** lorsque : si le séquent conclusion de la règle est valide, alors toutes les prémisses de la règle sont des séquents valides.

Cette notion correspond en fait à la réciproque de la définition de la correction d'une règle.

22. Illustrer avec un exemple que les règles $(\vee_i^g)$ et $(\vee_i^d)$ de la déduction naturelle ne sont pas inversibles.

23. Justifier brièvement pourquoi toutes les règles d'élimination de la déduction naturelle (sauf $(\neg_e)$) ne sont pas inversibles.
24. Montrer que la règle (Cut) du calcul des séquents n'est pas inversible.
25. Montrer que toutes les autres règles du calcul des séquents sont inversibles. *On pourra se contenter de le montrer pour $(\top)$, $(\wedge \vdash)$, $(\vdash \wedge)$ et $(\rightarrow \vdash)$ et admettre qu'un raisonnement similaire fonctionne pour les autres règles.*

La conséquence de ce résultat est que si un séquent est prouvable, et que plusieurs règles peuvent être appliquées à ce séquent : on peut choisir n'importe quelle règle à appliquer en premier, et les prémisses seront toujours prouvables !

26. Soit $\Gamma \vdash \Delta$ un séquent tel qu'aucune règle du calcul des séquents ne peut être appliquée.

a. Montrer que :

- Γ ne contient que des variables, et éventuellement $\top$;
- Δ ne contient que des variables, et éventuellement $\perp$;
- $\Gamma \cap \Delta = \emptyset$.

b. En déduire que $\Gamma \vdash \Delta$ n'est pas valide.

Un **arbre de preuve partiel** est un arbre dont chaque nœud interne correspond à une instance valide d'une règle du calcul des séquents. À la différence d'un « vrai » arbre de preuve, on ne demande pas que les feuilles soient des instances de règles sans prémisses.

27. Soit $\Gamma \vdash \Delta$ un séquent. Soit A un arbre de preuve partiel dont $\Gamma \vdash \Delta$ est le séquent à la racine.
 - a. Justifier que la hauteur de A est bornée par une valeur dépendant de Γ et Δ que l'on explicitera.
 - b. En déduire que le calcul des séquents est un système de preuve complet.